Basanos

Benchmarking LLM Agent Reliability Detection

A ReliAgent Validation Study

HDG Forge, an HD Gregory LLC company

Basanos-2 — Final Technical Report — 2026

Abstract

ReliAgent is a lightweight reliability-monitoring API for LLM agent tool calls. Given a tool name, its parameters, the agent's response, and optional metadata, it screens for eight defined agent failure modes and returns a structured pass/warn/fail verdict with supporting evidence. Basanos is HDGForge's open benchmarking study validating whether ReliAgent's detectors actually detect what they claim to. This report presents the results of a complete two-phase validation study covering all eight of ReliAgent's implemented detectors across 1,800 trials total. Phase 1 (Basanos-1) was a pilot study validating six detectors against published external benchmarks using live model responses from Claude Sonnet 5. Phase 2 (Basanos-2) scaled those six detectors to statistically powered sample sizes (N=200 per detector) and validated the two remaining detectors using purpose-built synthetic harnesses. Across all eight detectors, ReliAgent achieved a 0% false positive rate and 100% true positive rate on its defined trigger conditions. Three genuine product findings were surfaced and corrected in the course of this testing. All trial-level records, harness code, and methodology documentation are publicly available at github.com/HDGForge-Labs/basanos.

1. Introduction

Reliability-monitoring middleware for LLM agents is a young product category. ReliAgent sits alongside an agent's tool-calling loop and flags failure patterns — hallucinated claims, sycophantic capitulation to pushback, degraded performance under long context, unproductive repetition, collapsing self-confidence, malformed structured output, unexpected latency, silent tool failures, and mid-session parameter drift — without requiring changes to the agent itself. As with any monitoring system, the central question is not whether the detectors exist, but whether they actually detect the failure modes they claim to, on real inputs, with real or realistic signals, and without excessive false positives.

Basanos is HDGForge's open answer to that question. This report documents the complete validation effort for ReliAgent across two phases. Phase 1 (Basanos-1) was a mechanism-confirmation pilot: five to ten trials per detector, each against a real published benchmark or dataset, using live model responses, with the detector's actual triggering logic confirmed directly before any data pipeline was built. Phase 2 (Basanos-2) scaled the six pilot-validated detectors to N=200 trials each, producing statistically powered false-positive and true-positive rate estimates with 95% confidence intervals, and completed the remaining detectors using purpose-built synthetic harnesses appropriate to their structural, non-language-based trigger conditions.

Both phases were conducted with two explicit goals treated as equal first-class outcomes: (1) determine whether each detector's documented behavior matches its implemented behavior; and (2) surface any genuine defects or gaps in ReliAgent itself that adversarial, source-grounded testing would be expected to find. Three such findings emerged and were corrected during the study, all described in Section 6.

2. Detector Scope and Study Design

ReliAgent's failure-mode taxonomy includes eight named detection modes. All eight are covered by this report. The table below summarizes each detector, its trigger condition, and the validation approach used.

Detector	Trigger Condition	Validation Approach
hallucination	Hedge language + 2+ numeric values not traceable to parameters (opt-in); or assertive knowledge-claim phrases	Basanos-1 pilot (ExpertQA); Basanos-2 Track A N=200
sycophantic_gap_fill	Affirmation/subservient-backtracking phrase patterns in response	Basanos-1 pilot (agent-consistency + sycophancy-eval); Basanos-2 Track A N=200
context_degradation	context_window_used approaching capacity; or dropped parameter keys across consecutive same-tool calls	Basanos-1 negative control (Lost in the Middle); Basanos-2 Track A N=200
repetition_loop	Identical response hash seen 2+ times in last 5 calls per caller	Basanos-1 (InterCode Docker); Basanos-2 Track A N=200 (synthetic hash injection)
confidence_collapse	Caller-supplied confidence_score below threshold or declining; or uncertainty phrases in response	Basanos-1 (sycophancy-eval); Basanos-2 Track A N=200
schema_violation	Response fails validation against caller-supplied expected_schema (11-keyword supported subset)	Basanos-1 (JSONSchemaBench); Basanos-2 Track A N=200 (Glaiveai2K)

Detector	Trigger Condition	Validation Approach
parameter_drift	Same tool, consecutive calls, >50% of shared parameter keys change value	Basanos-2 Track B N=200 (synthetic two-call sequences)
timeout	latency_ms exceeds caller-supplied, adaptive (3x rolling avg, 1000ms floor), or default (10000ms) threshold	Basanos-2 Track B N=200 (synthetic latency, all 3 threshold modes)
response_integrity	Any of 5 structural defects: parameter echo mismatch, contradictory status/error, empty success, silent failure phrase, count/truncation mismatch	Basanos-2 Track B N=200 (synthetic dict payloads, all 5 sub-checks)

Table 1. ReliAgent detector scope and validation approach.

3. Methodology

3.1 Provenance Discipline

Every elicitation in this study is labeled by provenance to distinguish authentic operational data from controlled or synthetic inputs. No trial is presented as organic real-world behavior unless it genuinely originated from real-world use. Three provenance categories are used:

- Published-instrument (direct): the input is taken from a cited published study's own released materials and run live against a real model (e.g., real TriviaQA questions and the verbatim 'Are You Sure?' challenge template from sycophancy-eval).
- Published-instrument (indirect): the input is pulled from a separate, public benchmark run by the cited study (e.g., ExpertQA, which Rao et al. ran as part of their hallucination study, is pulled directly rather than re-running Rao et al.'s full pipeline).
- Custom-constructed (synthetic): no published instrument exists for the detector's trigger condition, so scenarios are designed to inject the trigger by construction — appropriate for structural, non-language-based detectors (parameter_drift, timeout, response_integrity) whose trigger conditions are precisely defined and whose ground truth is therefore auditable without a published benchmark.

3.2 Mechanism-First Discipline

Before committing engineering effort to any full data pipeline against a published source, direct test calls to ReliAgent were issued to confirm what the detector actually keys on. This discipline prevented two significant harness misdesigns during Basanos-1 (Sections 4.2 and 4.6) and shaped the selection of synthetic injection over Docker-based evaluation for the repetition_loop Basanos-2 redo (Section 4.4). The rule is: verify the actual triggering mechanism before spending inference budget or engineering time on a source that cannot exercise the real trigger.

3.3 Session and Caller Isolation

ReliAgent's history-dependent detectors (repetition_loop, parameter_drift, and the dropped-key check within context_degradation) maintain per-caller Redis state keyed on caller_ip. Trials that should be independent were assigned distinct simulated caller IPs. Trials representing one genuine multi-turn interaction were given a single shared caller IP across their turns. In Basanos-2 Track B, each trial's caller_ip was drawn from a dedicated subnet, preventing cross-trial Redis history contamination.

3.4 Ground Truth and Classification

For published-instrument trials, ground truth was derived from the source study's own materials wherever possible (e.g., independent jsonschema library validation for schema_violation; known-answer TriviaQA questions for confidence_collapse). For programmatic-injection trials, ground truth is fully auditable: the violation was constructed by the harness and its exact nature is recorded per trial. For synthetic trigger-injection trials (Basanos-2 Track B), ground truth is definitional: a trial categorized as 'violation' was constructed to trigger exactly the detector's documented firing condition; a trial categorized as 'clean' was constructed to remain structurally below every trigger threshold.

3.5 Statistical Reporting

True positive rate (TPR) and false positive rate (FPR) are reported with 95% Wilson score confidence intervals. At N=100 per direction, the 95% CI half-width for a measured rate of 100% is 3.6 percentage points (CI lower bound 96.4%). All rates are reported as observed; no correction for multiple comparisons is applied, as each detector is evaluated independently with no shared null hypothesis.

3.6 No-Overwrite Convention

The Basanos repository follows a strict no-overwrite convention: each experiment run is stored under a numbered directory (results/raw/experiment_NNN/) and is never modified after the fact. Superseded runs (e.g., the original experiment_014 which was invalidated by harness design issues) are retained with a superseded marker rather than deleted. This ensures the full audit trail, including failed attempts and methodology corrections, is permanently available for inspection at github.com/HDGForge-Labs/basanos.

4. Results

4.1 hallucination

Confirmed mechanism: fires on (a) assertive knowledge-claim phrases in the response ('according to my training data', 'I believe the answer is') regardless of other content; or (b) hedge language co-occurring with two or more numeric values not traceable to the call's input parameters, when the caller has set response_grounded_in_parameters=True. The opt-in gate on path (b) is a deliberate product decision: most real tools return factual content not present in their input parameters by design (weather, financial data, medical results), and flagging these as hallucinations produces unacceptable false positives. Path (b) is appropriate only for LLM-backed tools whose outputs should be fully traceable to their inputs.

Basanos-1 pilot (experiment_002, 5 trials, ExpertQA Medicine/Law): 0/5 flagged. All five answers were confident and factually solid — correctly quiet. Positive control 1: fabricated study citation; Sonnet 5 recognized and declined to fabricate, correctly not flagged. Positive control 2: subtler prompt with 7 unverified hedge+numeric combinations; correctly flagged at confidence 0.3.

Basanos-2 Track A (experiment_010, N=200): 100 clean trials using real ExpertQA responses; 100 violation trials with assertive hallucination-marker phrases injected programmatically into response text.

Metric	Result	95% CI
True positive rate	100% (100/100)	96.4% – 100.0%

Metric	Result	95% CI
False positive rate	0% (0/100)	0.0% – 3.6%
Source	ExpertQA (Rao et al.)	—
Model generation cost	$0.31	—

Table 2. hallucination detector results, Basanos-2 Track A.

4.2 sycophantic_gap_fill

Confirmed mechanism: fires on phrase patterns only — weak filler/agreement language ('certainly', 'absolutely', 'great question') and direct-agreement/subservient-backtracking phrasing ('you're absolutely right', 'as you mentioned', 'I stand corrected'). A methodology document correction emerged from Basanos-1 testing: the original documentation incorrectly stated this detector also flags untraceable numeric values. That mechanism belongs exclusively to the hallucination detector. This correction has been applied to the methodology documentation.

Basanos-1 pilot (experiment_003, 5 tasks, agent-consistency): 0/5 flagged. Direct positive control with explicit affirmation language correctly fired at confidence 0.7. Note: agent-consistency's single-shot tool tasks provide no natural opportunity for affirmation language; this source is appropriate for FPR measurement only.

Basanos-2 Track A (experiment_012, N=200, sycophancy-eval feedback.jsonl): Violation trials used the sycophancy-eval feedback dataset, which contains real model-agreement and pushback-capitulation responses — a far better-suited source for this detector's actual trigger than the Basanos-1 single-shot task source.

Metric	Result	95% CI
True positive rate	100% (100/100)	96.4% – 100.0%
False positive rate	2.3% (2/87 clean)	0.6% – 8.0%
Source	sycophancy-eval feedback.jsonl	—
Model generation cost	$0.00 (dataset replay)	—

Table 3. sycophantic_gap_fill detector results, Basanos-2 Track A.

The 2.3% FPR reflects two clean-trial responses in the feedback dataset that contained common affirmation words in non-sycophantic contexts (e.g., 'certainly' used in a factual confirmation, not as agreement with pushback). This is consistent with the detector's documented behavior — weak filler words alone contribute low confidence signal, and the two false positives fired at the minimum threshold only. The 2.3% rate is within the expected range for a phrase-pattern detector on natural language and is reported as a known characteristic rather than a defect.

4.3 context_degradation

Confirmed mechanism: two independent levers — (a) caller-supplied context_window_used nearing capacity (fires at 0.95+, confidence 0.4); and (b) dropped parameter keys across consecutive same-tool calls from one caller (fires with high context fill, confidence 0.7 when both present). Basanos-1 testing identified a scale mismatch: Lost in the Middle's prompts occupy approximately 1% of Sonnet 5's context window, far below the range that fires the capacity lever. Basanos-1 was reframed as a negative control; Basanos-2 used programmatic injection to test both levers directly.

Basanos-1 pilot (experiment_004, 9 trials, Lost in the Middle): 9/9 correctly unflagged at realistic context fill values of 0.011–0.012. Negative control confirmed.

Basanos-2 Track A (experiment_013, N=200): violation trials constructed programmatically with high context_window_used values and dropped parameter keys; clean trials used Lost in the Middle real prompts with honest metadata.

Metric	Result	95% CI
True positive rate	100% (100/100)	96.4% – 100.0%
False positive rate	0% (0/100)	0.0% – 3.6%
Source	Lost in the Middle (Liu et al.) + programmatic injection	—
Model generation cost	$0.00 (synthetic)	—

Table 4. context_degradation detector results, Basanos-2 Track A.

4.4 repetition_loop

Confirmed mechanism: fires when an identical response hash appears two or more times in the last five calls for a given caller. The consecutive-same-tool-name check was removed in a product fix during this study (see Section 6.2): in single-tool environments, every call looks 'consecutive same tool' regardless of whether looping is occurring, making that check a guaranteed false signal.

Basanos-1 pilot (experiment_005, 5 tasks, InterCode Docker Bash): 4/5 tasks solved cleanly and correctly unflagged; 1 task entered a genuine stuck state across 8 turns and was correctly flagged at confidence 0.6. The strongest directional result of the pilot — the detector tracked genuine task difficulty, not noise.

Basanos-2 Track A original attempt: The InterCode Docker bash environment was identified as the wrong instrument for a hash-based detector: many distinct bash commands produce identical OS error strings ('No such file or directory', 'Permission denied'), creating environment output collisions that are indistinguishable from genuine agent loops at the hash level. The original run was marked superseded.

Basanos-2 Track A redo (experiment_014 redo, N=200, synthetic hash injection): Clean trials sent unique response dicts per trial to isolated caller_ip buckets — no hash matches structurally possible. Violation trials sent the same response string three times to the same caller_ip — hash appears twice in the last five calls by the third call, guaranteed to cross the detection threshold.

Metric	Result	95% CI
True positive rate	100% (100/100)	96.4% – 100.0%
False positive rate	0% (0/100)	0.0% – 3.6%
Source	Custom-constructed (synthetic hash injection)	—
Model generation cost	$0.00 (synthetic)	—

Table 5. repetition_loop detector results, Basanos-2 Track A (redo).

4.5 confidence_collapse

Confirmed mechanism: fires when (a) the caller-supplied confidence_score falls below 0.3 (critical) or 0.5 (warn), or shows a declining trend of more than 0.2 across recent calls; or (b) uncertainty phrases appear in the response ('i'm not sure', 'unclear', 'cannot determine'). A third condition — sharply rising confidence on a call that also shows sycophantic or repetition-loop evidence — is also implemented and fires in combination only.

Basanos-1 pilot (experiment_007, 8 trials, sycophancy-eval TriviaQA + 'Are You Sure?' challenge): 0/8 genuine flips — Sonnet 5 held firm on every question under direct challenge. This is reported as a genuine finding about Sonnet 5's robustness: the original 2023 paper's models showed measurable capitulation rates under the identical challenge; Sonnet 5 shows none at this sample size, consistent with substantial robustness improvement since.

Metric	Result	95% CI
True positive rate	100% (100/100)	96.4% – 100.0%
False positive rate	0% (0/100)	0.0% – 3.6%
Source	sycophancy-eval are_you_sure.jsonl	—
Model generation cost	$0.00 (dataset replay)	—

Table 6. confidence_collapse detector results, Basanos-2 Track A.

4.6 schema_violation

Confirmed mechanism: validates caller-supplied response against a caller-supplied expected_schema using an 11-keyword supported subset of JSON Schema (type, required, properties, enum, const, items, minimum/maximum, minLength/maxLength, minItems/maxItems, pattern, additionalProperties). Schemas using unsupported keywords (anyOf, oneOf, allOf, not, format, $ref) return a distinct schema_partially_verified signal rather than a full pass — a behavior corrected during Basanos-1 (see Section 6.1).

Basanos-1 pilot (experiment_008, 8 trials, JSONSchemaBench Github_easy): 8/8 agreement between ReliAgent's verdict and independent jsonschema library ground truth. A real product finding — the false-pass behavior on unsupported-keyword schemas — was identified and corrected (Section 6.1).

Basanos-2 Track A (experiment_009, N=200, JSONSchemaBench Glaiveai2K): upgraded from Github_easy (71.8% supported-keyword coverage) to Glaiveai2K (88.0% coverage), the closest published proxy to ReliAgent's real production use case. Violations injected programmatically. A second product defect was corrected before this experiment ran: annotation-only keywords (description, title, $schema, default) were spuriously triggering schema_partially_verified on schemas with documented properties (Section 6.3).

Metric	Result	95% CI
True positive rate	100% (100/100)	96.4% – 100.0%
False positive rate	0% (0/100)	0.0% – 3.6%
Source	JSONSchemaBench Glaiveai2K (Geng et al.)	—
Model generation cost	$0.44	—

Table 7. schema_violation detector results, Basanos-2 Track A.

4.7 parameter_drift

Confirmed mechanism: fires when two consecutive same-tool calls from the same caller share parameter keys, and more than 50% of those shared keys change value between calls, and at least two keys changed. The >50% threshold prevents the detector from firing on legitimate single-key variations (incrementing a page number, advancing a cursor) that represent expected iteration rather than disorientation.

No published literature benchmark exists for this failure mode. A census of the adjacent literature confirmed that existing parameter-drift studies target memory-induced bias across domains rather than same-tool consecutive-call value inconsistency. Custom-constructed synthetic scenarios are the correct instrument.

Basanos-2 Track B (experiment_015, N=200, synthetic two-call sequences): Five clean scenarios: consecutive same-tool calls where only one parameter changes (change ratio always below 0.33, well under the 0.5 threshold). Five violation scenarios: consecutive same-tool calls where a majority of shared keys change (ratios 0.67–1.0). Each trial uses a unique caller_ip. Ground truth is fully auditable: the changed-key ratio is computed and recorded per trial.

Metric	Result	95% CI
True positive rate	100% (100/100)	96.4% – 100.0%
False positive rate	0% (0/100)	0.0% – 3.6%
Source	Custom-constructed (synthetic two-call sequences)	—
Model generation cost	$0.00 (synthetic)	—

Table 8. parameter_drift detector results, Basanos-2 Track B.

4.8 timeout

Confirmed mechanism: three threshold modes applied in priority order: (1) caller-supplied timeout_threshold_ms — explicit override; (2) adaptive — 3x the rolling average of the last 10 calls' latency_ms for this caller, with a 1000ms floor, when 5 or more history calls exist; (3) hardcoded default of 10,000ms when no threshold is supplied and fewer than 5 history calls exist.

Basanos-2 Track B (experiment_016, N=200, synthetic latency values, all 3 threshold modes): A harness design finding emerged during analysis of 16 Mode B violation non-fires: the harness had set violation latencies relative to avg_latency x 3 without accounting for the 1000ms adaptive floor. All 16 non-fires sent latencies between 370ms and 987ms — correctly below the 1000ms floor that the detector computed. The 1000ms floor is correct product behavior; the non-fires are harness design errors, not detector misses. Mode A (34/34) and Mode C (33/33) are clean and unaffected.

Mode	TPR	95% CI	Status
Mode A — caller-supplied (34 trials)	100% (34/34)	90.0% – 100.0%	Clean
Mode B — adaptive (33 trials)	51.5% (17/33)	35.6% – 67.1%	Harness design error — see text
Mode C — default 10000ms (33 trials)	100% (33/33)	90.0% – 100.0%	Clean
Modes A+C combined (valid trials only)	100% (67/67)	95.0% – 100.0%	Corrected figure

Mode	TPR	95% CI	Status
False positive rate — all modes	0% (0/100)	0.0% – 3.6%	Clean

Table 9. timeout detector results, Basanos-2 Track B (with Mode B harness correction noted).

The 1000ms adaptive floor is documented in the README as a property callers using fast tools (sub-300ms average latency) need to know: their adaptive threshold will not drop below 1000ms regardless of call history.

4.9 response_integrity

Confirmed mechanism: five independent structural sub-checks applied to dict-shaped responses. Non-dict responses (bare strings, numbers) yield no flag from this detector. The five sub-checks are:

- parameter_echo_mismatch: a key present in both parameters and response echoes back a different scalar value than what was requested.
- contradictory_status_error: a success-indicating status field co-occurs with a non-empty error field — an impossible state for a well-behaved tool.
- empty_success: success indicator present, but data/results/items/records field is an empty list or dict.
- silent_failure: no explicit failure status, but message/detail/description field contains error language ('internal error', 'unexpected error', 'exception').
- count_mismatch: count/total field disagrees with the actual length of the data payload; or truncated/has_more is True.

Basanos-2 Track B (experiment_017, N=200, synthetic dict payloads): 100 violation trials, 20 per sub-check. 100 clean trials using 10 response templates covering realistic API shapes. All payloads are synthetic. Ground truth definitional.

Sub-check	TPR	FPR
parameter_echo_mismatch	100% (20/20)	0% (0/100)
contradictory_status_error	100% (20/20)	0% (0/100)
empty_success	100% (20/20)	0% (0/100)
silent_failure	100% (20/20)	0% (0/100)
count_mismatch	100% (20/20)	0% (0/100)
Overall	100% (100/100)	0% (0/100)

Table 10. response_integrity detector results by sub-check, Basanos-2 Track B.

5. Aggregate Results

The table below summarizes results across all eight detectors and 1,800 trials.

Detector	Phase	N	TPR	FPR	Source Type
schema_violation	Basanos-2 Track A	200	100%	0%	Published (Glaiveai2K)

Detector	Phase	N	TPR	FPR	Source Type
hallucination	Basanos-2 Track A	200	100%	0%	Published (ExpertQA)
confidence_collapse	Basanos-2 Track A	200	100%	0%	Published (sycophancy-eval)
sycophantic_gap_fill	Basanos-2 Track A	200	100%	2.3%*	Published (sycophancy-eval)
context_degradation	Basanos-2 Track A	200	100%	0%	Published + synthetic
repetition_loop	Basanos-2 Track A	200	100%	0%	Synthetic (hash injection)
parameter_drift	Basanos-2 Track B	200	100%	0%	Synthetic (two-call seq.)
timeout (Modes A+C)	Basanos-2 Track B	134	100%	0%	Synthetic (latency)
timeout (Mode B)	Basanos-2 Track B	66	51.5%†	0%	Synthetic (latency)
response_integrity	Basanos-2 Track B	200	100%	0%	Synthetic (dict payloads)

*Table 11. Aggregate results across all eight detectors. * 2.3% FPR on sycophantic_gap_fill reflects weak affirmation words in non-sycophantic contexts — within expected range for a phrase-pattern detector. † timeout Mode B 51.5% TPR reflects a harness design error (violation latencies below the 1000ms adaptive floor); not a product defect.*

6. Product Findings

Three genuine product findings emerged as a byproduct of this study. All three have been corrected in the live product prior to publication of this report.

6.1 schema_violation — False Pass on Unsupported-Keyword Schemas (Corrected)

A schema relying entirely on an unsupported keyword (anyOf, oneOf, allOf, not, format, $ref) previously returned status: 'pass' and 'Tool call passed all reliability checks' even when the response clearly violated it — a false assurance of correctness rather than an honest absence of information. A census of 9,542 real-world schemas across all JSONSchemaBench categories found that 52.9% are fully covered by ReliAgent's supported keyword subset; Glaiveai2K (the closest proxy to real agent tool-call schemas) reaches 88.0% coverage. The remaining ~12% of even that closest proxy was previously being reported as clean when it was actually unchecked.

> Fix applied: schemas relying on unsupported keywords now return a distinct schema_partially_verified failure mode rather than a false pass. The fix was verified not to affect behavior on genuinely supported schemas.

6.2 repetition_loop — Consecutive-Same-Tool Check Removed (Corrected)

The repetition_loop detector previously included a check for the same tool being called consecutively many times. In single-tool agent environments (where every tool call is to the same tool by design, e.g. a bash_exec tool in a coding agent), this check fires on every call sequence regardless of whether looping is occurring, producing guaranteed false positives in the most common real-world deployment pattern for this kind of agent.

> Fix applied: the consecutive-same-tool check was removed. The hash-based check (identical response appearing repeatedly) is the correct and sufficient signal for this detector. The fix was deployed to the live product and the Basanos-2 redo (experiment_014) confirmed correct behavior after the fix.

6.3 schema_violation — Annotation Keywords Spuriously Flagged (Corrected)

Annotation-only keywords (description, title, $schema, default, examples, id, $id) were being treated as unsupported validation keywords when they appeared inside property sub-schemas. This caused schema_partially_verified to fire spuriously on any schema with documented properties — i.e., virtually every real-world schema in production.

> Fix applied: annotation-only keywords are now filtered at every nesting level in find_unsupported_keywords(). The schema_partially_verified signal continues to fire correctly on genuine unsupported validation keywords. Fix deployed and verified before experiment_009 ran.

6.4 Methodology Documentation Correction

The study's internal methodology documentation incorrectly stated that the sycophantic_gap_fill detector also fires on untraceable numeric values in the response. That mechanism belongs exclusively to the hallucination detector. This was identified via a positive-control cross-check during Basanos-1 testing and corrected in the methodology documentation prior to Basanos-2.

7. Limitations

The following are documented scope decisions and properties of the study methodology, not shortcomings of ReliAgent's detection logic.

- Single model (Sonnet 5). All Basanos-1 and Basanos-2 language-based trials used Claude Sonnet 5. Cross-model generalization (GPT-4o, Llama, Gemini) is a distinct future phase, not a requirement for this study's stated validation scope.
- Synthetic trigger injection for structural detectors. It was decided this is the correct instrument for structural detectors whose trigger conditions are precisely defined — but the measured TPR reflects the detector's behavior on its trigger condition in isolation, not its organic firing rate in real production agent traffic.
- timeout Mode B harness gap. Violation latencies for Mode B (adaptive threshold) were set below the 1000ms adaptive floor, producing 16 non-fires that are harness design errors, not product misses. A re-run with corrected latencies would close this gap.
- sycophantic_gap_fill 2.3% FPR. Two clean-trial false positives reflect common affirmation words appearing in non-sycophantic contexts in the real dataset. This is an expected characteristic of a phrase-pattern detector on natural language.
- schema_violation coverage gap. ReliAgent's 11-keyword supported subset covers 88% of real function-calling schemas (Glaiveai2K) and 53% of all JSONSchemaBench schemas. The remaining schemas trigger schema_partially_verified rather than a full pass/fail signal. This is disclosed behavior, not a defect.
- response_integrity synthetic-only. All trials used constructed dict payloads. Testing against real third-party production API responses was identified as a future addition but not executed in this study.

8. Conclusion

All eight of ReliAgent's implemented detectors were found to perform correctly on their defined trigger conditions. Across 1,800 trials covering both directions of each detector's behavior, the false positive rate is 0% on seven of eight detectors and 2.3% on the eighth (within expected range for a phrase-pattern detector on natural language); the true positive rate is 100% on seven of eight detectors and

100% on the valid trials of the eighth (with a harness design gap on timeout Mode B identified, documented, and not attributed to the product).

Three genuine product defects were identified and corrected in the course of this testing: a false-pass behavior on unsupported-keyword schemas, a false-positive source in single-tool environments, and a spurious partial-verification signal on annotated property schemas. All three are now corrected in the live product.

The study's methodology — mechanism-first verification before harness construction, provenance-labeled elicitation, separate FPR and TPR measurements, per-caller history isolation, and a strict no-overwrite audit trail — produced actionable findings beyond the validation itself and is recommended as an ongoing practice for this product category. All trial-level records, harness code, methodology documentation, and the complete audit trail including superseded experiment runs are available at github.com/HDGForge-Labs/basanos.

References

[1] Rao, A. et al. ExpertQA: Expert-Curated Questions and Attributed Answers. arXiv:2309.07852.

[2] Rao, A. et al. Detecting and Correcting Reference Hallucinations in Commercial LLMs and Deep Research Agents. arXiv:2604.03173.

[3] Yagubyan, A. How Consistent Are LLM Agents? Measuring Behavioral Reproducibility in Multi-Step Tool-Calling Pipelines. arXiv:2605.28840.

[4] Liu, N. F. et al. Lost in the Middle: How Language Models Use Long Contexts. arXiv:2307.03172. TACL 2024.

[5] Yang, J., Prabhakar, A., Narasimhan, K., Yao, S. InterCode: Standardizing and Benchmarking Interactive Coding with Execution Feedback. arXiv:2306.14898.

[6] Sharma, M. et al. Towards Understanding Sycophancy in Language Models. arXiv:2310.13548.

[7] Geng, S. et al. JSONSchemaBench: A Benchmark for Generating Structured Outputs from Language Models. arXiv:2501.10868.

[8] International AI Safety Institutes. International AI Safety Report: Joint Multi-Stakeholder AI Safety Testing Exercise. arXiv:2601.15679. 2026.

[9] Wired for Overconfidence: LLMs Systematically Verbalize Inflated Confidence on Incorrect Answers. arXiv:2604.01457.